
KernelBlaster: In-Context Reinforcement Learning for Cross-Task Hardware-Aware Optimization

Kris Shengjun Dong^{1,2,*}, Dima Nikiforov², Sahil Modi¹, Sana Damani¹,
Edward Lin¹, Siva Kumar Sastry Hari¹, Christos Kozyrakis¹
¹NVIDIA, ²University of California, Berkeley

Abstract

Optimizing accelerator code across hardware architectures remains difficult because peak performance depends on target-specific choices in data movement, parallel decomposition, memory-hierarchy use, and instruction-level execution. Existing large language model (LLM)-based code-optimization agents can generate high-performing kernels, but they typically behave as task-local search procedures: they perform repeated naive rewrites, evaluate them, and discard most optimization experience after the task ends. We propose KERNELBLASTER, a Memory-Augmented In-Context Reinforcement Learning (MAIC-RL) framework that enables LLM coding agents to make systematically informed and reward-guided decisions. KERNELBLASTER stores target-specific optimization insights and measured outcomes in a retrievable *Knowledge Base*, which demonstrates its ability to adapt to code-optimization tasks and targets in different execution environment without updating model weights. We instantiate KERNELBLASTER across heterogeneous targets, including CUDA C++ on NVIDIA GPUs, OpenCL C on Qualcomm GPUs, and RISC-V Vector (RVV) intrinsic code on RISC-V vector cores, demonstrating that the same memory-augmented optimization loop can operate across different languages, instruction-set architectures, and hardware backends. Compared with a PyTorch baseline, KERNELBLASTER achieves geometric-mean speedups of $1.43\times$, $2.50\times$, and $1.50\times$ on KernelBench Levels 1, 2, and 3, respectively, and $3.65\times$ on SOL-ExecBench Level 1; on Qualcomm Adreno GPUs, KERNELBLASTER achieves a $5.8\times$ geometric-mean speedup over the QNN baseline.

1 Introduction

The growing compute demand of machine-learning applications has driven the use of highly parallel compute units for efficiency and performance. At the same time, emerging deep neural network (DNN) architectures and dynamic workloads introduce execution patterns that invalidate previously tuned kernels, forcing software stacks to continually maintain specialized kernel libraries for new operators and hardware targets [4, 23]. This process requires substantial engineering effort and domain expertise, while the rapid obsolescence of optimized kernels limits the use of emerging hardware capabilities and constrains achievable performance [21].

Large language model (LLM) agents have shown promise as code optimizers for accelerator kernels and compiler workflows [3, 7–9, 17, 24, 25]. However, most existing systems still behave as task-local search procedures: an agent samples candidate implementations for one benchmark, observes execution feedback, revises the code, and discards much of the resulting experience when the task ends. This design can improve kernels on a single backend, but it under-specifies cross-target optimization, where the agent must reason over backend-independent concepts such as locality, occupancy, tiling, vectorization, fusion, and redundant-memory elimination while emitting backend-specific code and interpreting backend-specific profiling signals. Existing search, prompt-optimization, and

evolutionary methods retain some optimization signal, but they generally lack an executable, persistent memory of profile-grounded state-action-outcome evidence that transfers across kernels and hardware targets.

We introduce **KERNELBLASTER**, a *Memory-Augmented In-Context Reinforcement Learning* (MAIC-RL) framework for cross-task code optimization. **KERNELBLASTER** treats the LLM context and retrieved memory entries as inference-time policy parameters. Instead of updating model weights, **KERNELBLASTER** updates a persistent *Knowledge Base* containing compact records of performance states, optimization actions, target constraints, textual critiques, and observed outcomes. During each rollout, an actor generates code transformations conditioned on retrieved memory. A critic evaluates the resulting trajectory using correctness, scalar performance, profiling evidence, and natural-language performance-gap analysis, then updates memory with both successful and unsuccessful experience. This process converts profiling feedback into text-based reward signals that teach the agent which assumptions held, which failed, and which transformations should be reused, constrained, or avoided.

We study this problem in accelerator-code optimization, a use-inspired domain where rewards are observable but sparse, delayed, and hardware-dependent. To test the generality of the methodology, we instantiate **KERNELBLASTER** across code-generation and code-optimization tasks targeting different instruction-set architectures (ISAs), programming languages, and hardware architectures: CUDA C++ on NVIDIA graphics processing units (GPUs), Open Computing Language (OpenCL) C on Qualcomm GPUs, and RISC-V Vector (RVV) intrinsic code on RISC-V vector cores. Although these targets differ in execution model and profiling interface, they expose common optimization structure that can be represented through states, actions, rewards, and textual gradients.

Our primary contributions are:

1. **Memory-augmented in-context reinforcement learning.** We formulate agentic code generation and optimization as in-context reinforcement learning over program states and transformation actions, where the mutable policy is a retrieved, persistent *Knowledge Base* rather than model weights.
2. **An actor-critic agent architecture for policy improvement.** We introduce a reward representation that combines scalar execution outcomes with natural-language critiques derived from compilation, correctness, profiling, and performance-gap evidence. These critiques act as textual gradients that update memory and guide future action selection.
3. **A compact, reusable long-term memory in a structured database.** We aggregate experience from prior optimizations into a structured *Knowledge Base* that maps performance states to optimization actions, measured outcomes, textual critiques, and target constraints. This backend-agnostic representation accumulates target-specific domain knowledge.
4. **Cross-task and cross-target adaptivity.** We evaluate the same learning loop across CUDA/NVIDIA, OpenCL/Qualcomm, and RVV/RISC-V targets. The results use hardware performance as evidence that the memory and textual-gradient mechanism transfer across languages, ISAs, and hardware platforms, rather than as a CUDA-specific speedup table alone. The full code base, including the agentic pipeline, baseline kernels, initialized databases, and test harnesses, has been open-sourced¹

2 Related Work and Background

Agentic systems for code optimization can be broadly categorized as shown in Figure 1. We focus on inference-time methods that avoid model-weight updates and use frontier LLMs with tool feedback. The main distinction among prior methods is what adapts across attempts: static prompting retains hand-written heuristics, search retains candidate trajectories within a task, prompt optimization revises textual instructions, and evolutionary systems retain candidate programs.

Static prompting. Early LLM-based optimization systems rely on fixed prompts augmented with manually engineered heuristics [8, 19, 27, 30]. These systems demonstrate that LLMs can assist in low-level code generation, but their optimization behavior is mostly fixed by the prompt. When the target architecture, toolchain, or workload changes, the prompt often must be redesigned by a domain expert. This limits adaptation and re-usability of the experience learned during inference.

¹An anonymous mirror of the full code base is released at <https://anonymous.4open.science/r/KernelBlaster> to respect the double-blind procedure.

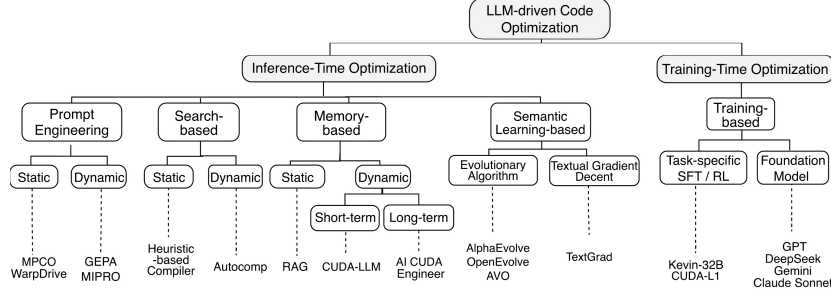


Figure 1: Taxonomy of agentic workflows for LLM-driven code optimization. KERNELBLASTER is positioned as a memory-augmented inference-time learning method: unlike static prompting and task-local search, it stores state-action-outcome evidence and uses textual gradients to update persistent policy memory across tasks.

Search-based optimization. Search-based systems generate multiple candidate kernels and select high-performing variants using compilation, testing, and timing feedback [6, 11]. Beam search and related pruning strategies improve sample efficiency relative to exhaustive enumeration. However, the resulting learning is usually task-local: each new kernel triggers another search process, and the system has limited ability to transfer bottleneck-level lessons from one task to another.

Evolutionary coding agents. Evolutionary systems such as AI CUDA Engineer [14], KernelEvolve [15], AlphaEvolve [18], CodeEvolve [2], and EvoEngineer [10] maintain populations of candidate programs and improve them through LLM-generated variation and evaluator feedback. These systems support broad exploration, but their memory is often represented as full program artifacts or elite candidates. Such archives are useful for reusing implementations, but they can obscure why an optimization worked. KERNELBLASTER instead stores compact, bottleneck-level state-action-outcome entries, including negative evidence, so retrieval can be driven by performance state rather than only source-code similarity.

Textual gradients and prompt-level learning. Prompt-optimization methods treat prompts as mutable textual parameters. GEPA performs reflective prompt evolution and Pareto-based selection [11]; TextGrad introduces the use of natural-language feedback as an analogue of gradients over textual variables [28]. Existing textual-gradient and prompt-optimization methods are primarily designed for task-local adaptation: a prompt or instruction set is optimized for a specific task distribution, toolchain, or evaluation environment and often must be re-optimized when those conditions change. This limits transfer across systems tasks, where the same high-level optimization concept may appear under different programming languages, hardware backends, and profiling interfaces. KERNELBLASTER builds on the textual-gradient idea but changes the optimization target: instead of updating only a task-specific prompt, textual gradients update a persistent *Knowledge Base* of state-action-outcome records that can be retrieved and reused across later code-optimization tasks.

Autotuning Frameworks. Compilers’ Kernel-level autotuning such as PyTorch 2’s TorchInductor further exposes `torch.compile` with `max-autotune`, TVM [5], Ansor [31], and Triton [26] profiles multiple template-based implementations and selects the fastest. LLM-driven profile-based autotuning frameworks use analytical models, template code generation, and search to discover backend-specific kernels. WaveTune [29] uses wave-aware performance models for low-latency tuning on fixed kernel sets; KernelBand [22] amortizes search with hierarchical bandits and profiling-aware clustering; and AutoKernel [12] combines edit-benchmark loops with curated playbooks and correctness harnesses. These methods are complementary to KERNELBLASTER: they target deep single-backend tuning, while KERNELBLASTER targets broader operator coverage and cross-target reuse through persistent semantic state-action memory. STACKFEED also uses text as gradient in an actor-critic loop, but edits retrieval knowledge for question answering. In contrast, AC-TGD updates a hardware-measured, bottleneck-level action memory for code optimization [13].

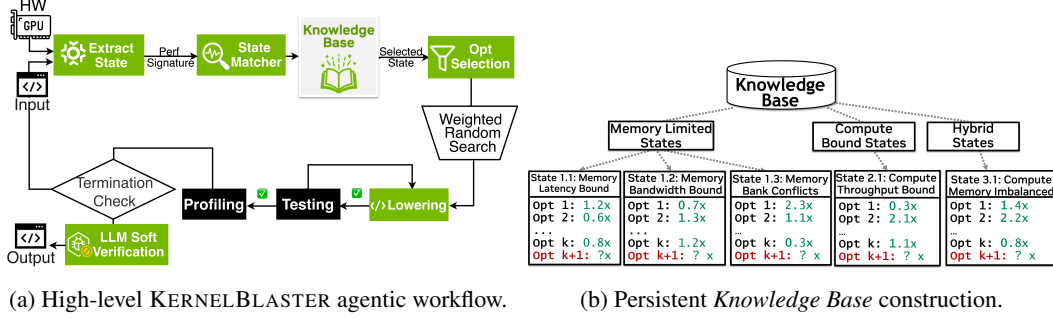


Figure 2: System overview. (a) The High-level block diagram of the KERNELBLASTER agentic workflow with memory-augmented optimization loop. (b) A performance signature is distilled into a state and associated with optimization actions, constraints, scalar outcomes, and textual critiques; the same memory schema is reused across all targets.

Table 1: Problem formulation for KERNELBLASTER’s Memory-Augmented In-Context Reinforcement Learning (MAIC-RL) loop.

Component	Definition
Task	$\tau = (x, y, b, m)$, where x is the input program or model fragment, y is a reference implementation, b is the backend, and m is the measurement interface.
State	$s_t = \phi(x_t, c_t, p_t, h_t)$, where x_t is the current implementation, c_t is compiler and validation feedback, p_t is timing or profiling evidence, and h_t is rollout history.
Actor action	$a_t \sim \pi_\theta(a \mid s_t, \mathcal{K}_t)$, where a_t is a code transformation or implementation plan and $\mathcal{K}_t$ is the set of retrieved <i>Knowledge Base</i> entries.
Reward signal	$(r_t, z_t) = R(x_{t+1}, y, b, m)$, where r_t is the correctness-gated scalar performance reward and z_t is the text-based reward signal.
Memory update	$\mathcal{K}_{k+1} = U(\mathcal{K}_k, \tau, \{(s_t, a_t, r_t, z_t)\}_{t=0}^T)$, where the critic incorporates rollout outcomes into persistent long-term memory.

3 Memory-Augmented In-Context Reinforcement Learning

KERNELBLASTER optimizes code through inference-time reinforcement learning with persistent memory. The method treats each optimization problem as an environment interaction: the agent observes a program state, samples an optimization action, lowers the action into executable code, receives scalar and textual feedback from validation and profiling, and updates a persistent *Knowledge Base*. Unlike model fine-tuning, this process does not change model weights. The policy improvement occurs through memory retrieval and memory rewriting.

3.1 Problem Formulation

We formulate KERNELBLASTER as an inference-time policy-improvement process over code-optimization tasks. Let an optimization task be $\tau = (x, y, b, m)$, where x is the input program or model fragment, y is a reference implementation, b is the backend or execution environment, and m is the measurement interface. At each rollout step, the actor proposes a transformation using the current state and retrieved memory, while the critic converts correctness and performance evidence into scalar and text-based rewards and updates the persistent *Knowledge Base*. Table 1 summarizes the notation.

The scalar reward r_t captures correctness-gated performance, while the text-based reward z_t explains the performance gap, failure mode, violated assumptions, and target-specific constraints. Thus, the policy parameters available to the LLM at inference time are not only the prompt template, but also the retrieved and updated persistent *Knowledge Base*.

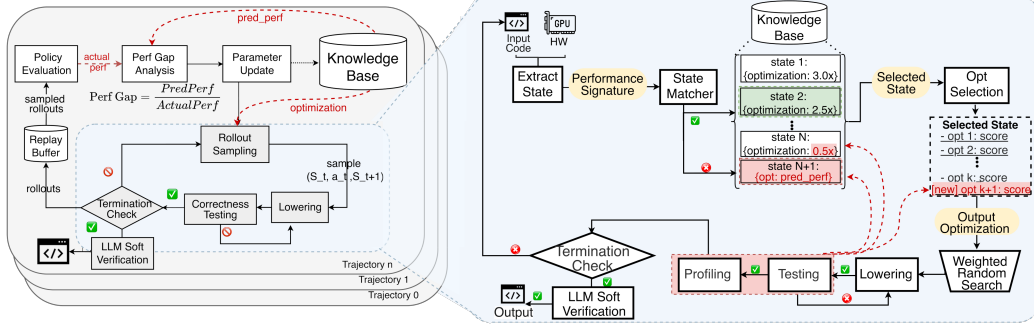


Figure 3: KERNELBLASTER as actor-critic inference-time reinforcement learning. The right sub-diagram is the Actor-Tactician, and the left sub-diagram is the outer Critic-Strategist.

3.2 Persistent Memory Representation

The *Knowledge Base* stores compact entries of the form

$$e_i = \langle s_i, (a_i, q_i, z_i, c_i) \rangle,$$

where s_i is a performance state, a_i is an optimization action, q_i is the measured or predicted score, z_i is a textual critique, and c_i records constraints such as backend assumptions, numerical requirements, or failure conditions. This representation is intentionally smaller than storing full programs. It is designed to answer the question: *given this bottleneck and target context, which transformations have historically helped or failed, and why?*

Memory entries are retrieved by matching the current performance state against prior states rather than only by matching source-code syntax (Figure 2b). This allows two kernels with different surface forms to share an optimization if their bottlenecks are similar. The memory also retains negative examples. Failed transformations are useful because they identify constraints under which a previously successful action should not be reused.

Table 2: Components of the KERNELBLASTER agentic workflow.

Component	Functionality
Knowledge Base	Retrievable store of entries of the form $\langle \text{State}, (\text{Optimization}, \text{Score}) \rangle$.
State Extractor	LLM-powered module that derives a performance signature from source code, compiler output, runtime timing, and available profiling counters.
State Matcher	Retrieves related states and optimization actions from the <i>Knowledge Base</i> .
Optimization Selector	Proposes candidate transformations and performs weighted sampling over high-reward candidates.
Backend Lowering Agent	Implements the selected transformation in CUDA, OpenCL, or RVV code.
Validation & Policy Evaluation	Checks correctness, measures performance, analyzes performance gaps, and updates the <i>Knowledge Base</i> .

3.3 Actor-Critic Textual Gradient Descent

KERNELBLASTER instantiates MAIC-RL as *Actor-Critic Textual Gradient Descent* (AC-TGD), a hierarchical inference-time learning architecture in which natural-language critiques serve as policy-update signals. AC-TGD follows the REINFORCE-like ICRL procedure in Algorithm 2; instead of updating model weights by gradient descent, KERNELBLASTER treats the LLM context and retrieved *Knowledge Base* entries as mutable policy parameters θ . The policy generates code transformations, receives rewards from compilation, correctness, timing, and profiling feedback, and updates persistent memory through natural-language performance-gap analysis. A high level overview of the MAIC-RL flow is shown in Figure 3.

Actor-Tactician. The Actor-Tactician runs the inner-loop optimization. Conditioned on retrieved memory $\mathcal{M}_t$ and the current state s_t , it proposes an optimization action consisting of a code transformation, a structured implementation plan, and the backend lowering needed to make the candidate

runnable. It handles local implementation decisions such as fusion, tiling, layout changes, vectorization, and backend-specific primitive selection.

Critic-Strategist. The Critic-Strategist runs the outer-loop policy update. It evaluates completed trajectories, compares expected and observed outcomes, identifies the reason for any discrepancy, decides which kernels or program states merit continued optimization, determines when trajectories should terminate, and selects which lessons should persist beyond the current task. This role is analogous to policy evaluation and advantage estimation, but operates in a textual rather than purely numerical space.

Text-based reward signals. The critic converts rollout evidence into a text-based reward signal z_t that augments the scalar reward r_t . A speedup value tells the agent whether a candidate improved, but not the reason or the cause of failure. In KERNELBLASTER, z_t is a structured critique derived from compilation logs, correctness checks, timing results, profiling counters, and the expected effect of the selected action. This critique records bottlenecks, failed assumptions, and reuse conditions in a form that can be retrieved by closed-weight LLMs, allowing KERNELBLASTER to update the next rollout by revising memory rather than training model weights.

Policy update. We implement the textual policy update through three modules. PolicyEvaluation compares achieved performance with the expected effect of the selected optimization. PerfGapAnalysis explains success or failure under the current backend and hardware constraints. ParameterUpdate revises memory by adjusting action scores, adding target-specific constraints, recording negative examples, or refining the state description.

Persistent long-term memory. The critic writes each textual gradient into two persistent memory structures: the operational *Knowledge Base* of state-action-outcome entries and a longer-form *GuideBook* of strategies, constraints, and known failure modes. Together, these structures form AC-TGD’s persistent long-term memory. Because the update is expressed in language, it can perform semantic credit assignment over multi-step trajectories, identify which step caused a regression, retain both successful and unsuccessful experience, and propagate lessons to future CUDA, OpenCL, and RVV tasks even when their surface code differs.

3.4 Cross-Task Instantiation Across Heterogeneous Targets

KERNELBLASTER generalizes beyond CUDA code optimization. To test whether the same persistent memory and text-based reward mechanism support cross-task and cross-domain reasoning, we instantiate the same actor-critic loop on code generation and code optimization problems whose targets differ in instruction-set architecture (ISA), source language, and hardware execution model: CUDA C++ on NVIDIA GPUs, OpenCL C on Qualcomm GPUs, and RVV intrinsic code on RISC-V vector cores. We treat all three as instances of a single methodology, sharing the same MAIC-RL/AC-TGD learning core and *Knowledge Base* schema. The Actor-Tactician’s prompt schema, the Critic-Strategist’s textual-gradient format, and the memory-retrieval mechanism are shared across targets; only the lowering, compilation, validation, and profiling components are backend-specific. This isolates the learning contribution from any given target’s engineering details. Backend-specific transformations the agent discovers (Tensor Core utilization, MMA layout, software warp specialization) with generated code samples in Appendix C.

Table 3: Analogy between classical policy-gradient reinforcement learning and KERNELBLASTER.

RL component	KERNELBLASTER analogue
Policy $\pi_\theta(a_t s_t)$	LLM actor conditioned on prompt context and retrieved <i>Knowledge Base</i> entries.
State s_t	Current program plus compiler, validation, profiling, and rollout-history evidence.
Action a_t	Code transformation, implementation plan, or backend-specific lowering decision.
Reward (r_t, z_t)	Correctness-gated scalar speedup plus a text-based critique explaining the outcome.
Parameters θ	Inference-time context: prompt, retrieved memory, and persistent <i>Knowledge Base</i> entries.
Policy evaluation	Critic summarizes trajectory outcomes, identifies performance gaps, and decides whether further optimization is worthwhile.
Policy update	Critic rewrites or adds <i>Knowledge Base</i> entries to change future retrieval and action selection.

Algorithm 1: REINFORCE

```

1: Input: Initial policy params  $\theta_0$ , env.  $\mathcal{E}$ , policy  $\pi_\theta$ 
2: Initialize replay buffer  $\mathcal{D} \leftarrow \emptyset$ 
3: for iteration  $k = 0, 1, 2, \dots$  do
4:   Sample initial state  $s_0 \sim \mathcal{E}$ 
5:   Initialize trajectory  $\tau \leftarrow []$ 
6:    $s \leftarrow s_0$ 
7:   for step  $t = 0$  to  $T$  do
8:     Sample action  $a_t \sim \pi_{\theta_k}(a_t | s_t)$ 
9:     Append  $(s_t, a_t)$  to trajectory  $\tau$ 
10:    Set  $s_{t+1} \leftarrow \text{EnvStep}(s_t, a_t)$ 
11:    Set  $r_t = R(s_t, a_t)$ 
12:  end for
13:  Evaluate return  $R(\tau) = \sum_t r_t$ 
14:  Store  $(\tau, R(\tau))$  in buffer  $\mathcal{D}$ 
15:  Estimate policy gradient:
16:   $\nabla_\theta J(\theta_k) \leftarrow \frac{1}{|\mathcal{D}|} \sum_{(\tau, R)} \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) R(\tau)$ 
17:  Update policy:  $\theta_{k+1} \leftarrow \theta_k + \alpha \nabla_\theta J(\theta_k)$ 
18: end for

```

Algorithm 2: LLM-Based Policy Opt. via Reward-Guided Rollouts

```

1: Input: Initial Knowledge Base  $\theta_0$ , env.  $\mathcal{E}$ , LLM policy  $\pi_\theta$ 
2: Initialize replay buffer  $\mathcal{D} \leftarrow \emptyset$ 
3: for iteration  $k = 0, 1, 2, \dots$  do
4:   Sample code task  $s_0 \sim \mathcal{E}$   $\triangleright$  Initial code
5:   Initialize rollout trajectory  $\tau \leftarrow []$ 
6:    $s \leftarrow s_0$ 
7:   for step  $t = 0$  to  $T$  do
8:     Generate optimized code  $a_t \sim \pi_{\theta_k}(a_t | s_t)$ 
9:     Append  $(s_t, a_t)$  to trajectory  $\tau$ 
10:    Set  $s_{t+1} \leftarrow a_t \triangleright$  Set new code as state
11:    Set  $r_t = R(s_{t+1}) \triangleright$  Profile-based reward
12:  end for
13:  Evaluate total reward  $R(\tau)$ 
14:  Store  $(\tau, R(\tau))$  in buffer  $\mathcal{D}$ 
15:   $g_k \leftarrow \text{PolicyEvaluation}(\mathcal{D}, \theta_k)$ 
16:   $p_k \leftarrow \text{PerfGapAnalysis}(g_k)$ 
17:   $\theta_{k+1} \leftarrow \text{ParameterUpdate}(\theta_k, p_k)$ 
18: end for

```

Table 4: KERNELBLASTER’s performance in driving CUDA, OpenCL across GPUs of different vendors and generations. **Top:** CUDA on NVIDIA GPUs - L40S and H100, baseline PyTorch. **Bottom:** OpenCL on Qualcomm Adreno 650, baseline QNN. **Color coding:** green = kernels generated by KERNELBLASTER outperform the baseline; red = underperform; yellow = comparable (within noise).

Level	Method	L40S								H100							
		Valid	Mean	GeoM	Med	Min	Max	%>1x	%<1x	Valid	Mean	GeoM	Med	Min	Max	%>1x	%<1x
1	CUDAEng	82%	1.728	1.101	1.202	0.0659	32.35	74.39%	25.61%	82%	1.935	1.025	1.085	0.0492	54.40	64.20%	35.80%
	Ours	93%	4.502	1.080	1.037	0.0801	183.96	59.72%	40.28%	86%	2.817	1.497	1.137	0.0647	32.74	70.37%	29.63%
2	CUDAEng	83%	3.865	1.695	1.574	0.328	123.39	79.52%	20.48%	82%	1.356	1.214	1.170	0.402	3.651	61.33%	38.67%
	Ours	95%	9.419	2.214	2.074	0.0488	362.29	72.60%	27.40%	81%	10.223	2.592	2.291	0.111	213.65	84.85%	15.15%
3	Ours	67%	1.749	1.502	1.894	0.529	2.681	75.0%	25.0%	67%	1.336	1.110	1.333	0.375	2.302	75.0%	25.0%
1	Ours	Qualcomm Adreno 650								43%	17.57	5.800	9.180	0.0142	73.77	85.00%	15.00%

4 Evaluation

We organize our cross-target and baseline comparisons around four research questions: **Q1.** Are the resulting implementations competitive with existing baselines? **Q2.** Does persistent memory improve cross-task optimization? **Q3.** Do text-based reward signals carry information beyond a scalar reward? **Q4.** Does the same memory schema transfer across heterogeneous targets?

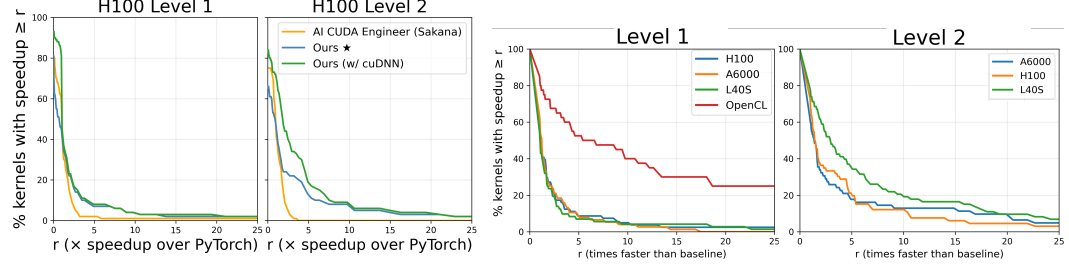
4.1 Experimental Setup

We evaluate KERNELBLASTER on code-optimization tasks drawn from KernelBench Levels 1–3 [20] and SOL-ExecBench [16]. CUDA/NVIDIA provides the largest quantitative comparison against PyTorch and prior CUDA-focused agentic systems. OpenCL/Qualcomm and RVV/RISC-V provide cross-target tests: they replace the target language, compiler, validation path, and measurement interface while preserving the same actor–critic loop and memory schema.

Across all environments, an optimization attempt is valid only if the generated code compiles, executes, and passes reference-based correctness checks. Performance is computed only for valid attempts. CUDA results use the better of `torch.eager` and `torch.compile` as the PyTorch baseline. Qualcomm results compare against the QNN SDK using per-operator GPU-event timing. RVV results compare against scalar C baselines under the target-side cycle measurement setup. The soft-verification agent we use to prevent shortcut behaviors (e.g., dispatching to PyTorch, eliminating profiled regions, calling external libraries) is described in Appendix B.2.

Table 5: Comparison among KERNELBLASTER (Ours) vs AI CUDA Engineer, against the PyTorch and the QNN SDK compiler baseline.

System	Language	Agent’s Models	Hardware Setup	Dataset	Hyperparameter	Test Harness
KERNELBLAS (Ours)	CUDA	GPT-5.0-mini, GPT-4.1, GPT-5.0, Sonnet 4.6, Opus 4.6	NVIDIA A6000, A100, H100, L40S; RISC-V Vector core (Saturn)	KernelBench L1–3, SOL-ExecBench	10 iterations, 10 rollout steps per iteration	Functionality Check: passing kernels are compiled and executed on GPUs; outputs are compared to PyTorch baselines with multiple randomized seeds to ensure correctness and prevent overfitting.
AI CUDA Engineer	CUDA	O4-mini, Sonnet 3.7, Gemini 2.5 Pro, GPT-4.1, O3	NVIDIA H100	KernelBench L1–2	10 generations; 10 proposals/generation; top 4 evaluated	LLM Soft-Verification: an LLM scans each kernel for likely compilation, memory, or numerical errors, filtering bad code before GPU execution.
QNN SDK Compiler	OpenCL	N/A	Qualcomm Adreno 650	KernelBench L1	N/A	(SDK-internal validation)



(a) H100: KERNELBLASTER vs. AI CUDA Engineer. (b) Cross-target: 3 NVIDIA GPUs + Qualcomm Adreno 650.

Figure 4: $fast_p$ curves comparing KERNELBLASTER to alternative agentic methods and across diverse GPU targets.

We provide a summary of our results on the primary KernelBench benchmarks in Table 4, showing speedup over the PyTorch baselines. Furthermore, we also demonstrate our portability across benchmarks, achieving $3.65\times$ speedup over PyTorch on SOL-ExecBench Level 1.

4.2 Evaluating Generality across Hardware Generations

CUDA/NVIDIA experiments provide the most mature benchmark and profiling interface. On KernelBench Level 1 and Level 2, more than half of the optimized solutions improve over the PyTorch baseline. Level 2 shows stronger gains because composed operators create more opportunities for multi-step transformations such as fusion, layout changes, and memory-hierarchy optimization. This supports the central claim that KERNELBLASTER is not only sampling isolated rewrites: it benefits from trajectories in which earlier transformations prepare the program for later transformations.

Compared with CUDA-focused agentic baselines, KERNELBLASTER achieves larger gains on Level 2 workloads, where cross-kernel optimization experience is most useful. The comparison is CUDA-specific because most public agentic baselines target CUDA/NVIDIA; we use the non-CUDA experiments to evaluate retargetability.

4.3 Evaluate Cross-Target Transferability

We evaluate KERNELBLASTER beyond CUDA by instantiating the same learning loop on OpenCL/Qualcomm and RVV/RISC-V targets, changing the programming language, execution model, and measurement interface while preserving the same state-action memory.

On 42 KernelBench Level 1 problems on a Qualcomm Adreno 650, KERNELBLASTER achieves a $5.80\times$ geometric-mean speedup over QNN, with a $9.18\times$ median speedup. The largest gains occur on reduction-axis operators, where the learned strategy maps a reduction bottleneck to cooperative local-memory reduction and vectorized global I/O in OpenCL. This suggests that the *Knowledge Base* stores a transferable reduction-state strategy rather than a CUDA-specific code template.

On RVV/RISC-V, KERNELBLASTER targets a vector programming model rather than a SIMT GPU model. It achieves $7.3\times$ speedup over a naive C baseline on convolutional neural network workloads such as ResNet and $2.98\times$ on small linear layers, using transformations such as unrolling, pipelining,

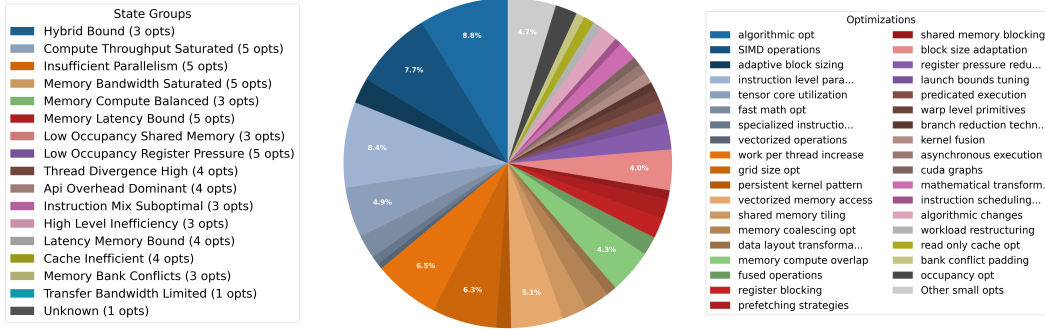


Figure 5: Distribution of 3972 CUDA optimization applications across performance states.

tiling, and algorithmic choices between `im2col` and direct convolution. These results show that the same memory-augmented agent remains useful when both the implementation language and hardware execution model change.

4.4 Adapting to Full-Model and Subgraph Optimization

We also evaluate whether the approach extends beyond isolated operators using KernelBench Level 3 models. In this setting, the agent operates over kernels that collectively form a full network or fused sub-block. The generated implementation for a Level 3 multi-layer perceptron (MLP) achieves 5.6x speedup over the PyTorch baseline, and a larger ResNetBasicBlock reaches 1.5x improvement. The relevant observation is not only the speedup, but the reuse pattern: Level 1 and Level 2 memory entries are applied to Level 3 through cross-layer fusion, activation reuse, and algebraic simplification. Per-model breakdowns and the corresponding generated code are provided in Appendix C.3.

Full-model optimization also exposes limits of the current approach. Whole-model code generation introduces long boilerplate, heterogeneous bottlenecks, and weaker performance signals. These results motivate hierarchical task decomposition, where the critic selects promising subgraphs or kernels for continued optimization rather than optimizing the entire model as a monolithic program.

4.5 Distribution of Optimizations learned in Persistent Memory

One key discovery of KERNELBLASTER is that the same persistent memory and text-based reward mechanism transfer across languages, ISAs, and hardware targets. We examine *Knowledge Base* composition to characterize what KERNELBLASTER actually accumulates as long-term memory and to show that the contents are reusable rather than task-specific.

The learned *Knowledge Base* is approximately 50 KB and stores optimization entries grouped by performance state. As Figure 5 shows, no state dominates the optimization process, with no state exceeding 20% of applications across 3,972 optimization applications on KernelBench L1+L2 (A6000). An average of 5.5 states is reached per kernel, showing that optimization changes the program’s measured behavior and that state assignment must be trajectory-aware rather than static.

The memory also captures useful transition structure. On L40S trajectories, repeated micro-tuning actions such as `instruction_level_parallelism` and `grid_size_optimization` often yield negligible improvements when applied repeatedly. In contrast, transitions between different optimization families expose higher-yield patterns. For example, applying `shared_memory_tiling` before `tensor_core_utilization` yields a median speedup of approximately 2.41x, while data-layout transformation before fusion yields approximately 1.95x. These patterns justify storing ordered state-action experience rather than only final candidate programs.

5 Conclusion

This paper presents KERNELBLASTER, an inference-time in-context reinforcement learning framework with persistent memory for agentic code optimization. Rather than building a CUDA-specific kernel generator, KERNELBLASTER converts execution feedback into reusable state-action knowl-

edge. It treats the LLM context and retrieved *Knowledge Base* entries as mutable policy parameters: an actor generates and lowers candidate transformations, while a critic converts correctness, profiling, and performance-gap evidence into text-based reward signals that update long-term memory.

Our results show that this mechanism improves optimization across tasks and targets. On CUDA/NVIDIA, KERNELBLASTER outperforms PyTorch and prior CUDA-focused agentic baselines, especially on Level 2 workloads where multi-step transformations are useful. On OpenCL/Qualcomm and RVV/RISC-V, the same memory schema and policy-update mechanism remain effective despite changes in programming language, ISA, hardware architecture, and profiling interface.

References

- [1] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gepa: Reflective prompt evolution can outperform reinforcement learning. Preprint, arXiv:2507.19457, 2025. URL <https://arxiv.org/abs/2507.19457>
- [2] Henrique Assumpção, Diego Ferreira, Leandro Campos, and Fabricio Murai. Codeevolve: An open source evolutionary coding agent for algorithm discovery and optimization. *arXiv preprint arXiv:2510.14150*, 2025. Open-source framework combining an islands-based genetic algorithm with modular LLM orchestration and execution feedback for selection and task-specific metrics.
- [3] Carlo Baronio, Pietro Marsella, Ben Pan, and Silas Alberti. Kevin-32b: Multi-turn rl for writing cuda kernels. <https://cognition.ai/blog/kevin-32b>, 2025. Cognition AI Blog. Accessed: May 19, 2025.
- [4] Siyuan Chen, Zhichao Lu, and Qingfu Zhang. Evolution of kernels: Automated RISC-V kernel optimization with large language models, 2025. URL <https://arxiv.org/abs/2509.14265>.
- [5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*, pages 578–594. USENIX Association, 2018.
- [6] Wentao Chen, Jiace Zhu, Qi Fan, Yehan Ma, and An Zou. Cuda-llm: Llms can write efficient cuda kernels. Preprint, arXiv:2506.09092, 2025. URL <https://arxiv.org/abs/2506.09092v1>.
- [7] Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. LLM compiler: Foundation language models for compiler optimization. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction*, pages 141–153, 2025.
- [8] Sana Damani, Siva Kumar Sastry Hari, Mark Stephenson, and Christos Kozyrakis. Warpdrive: An agentic workflow for ninja gpu transformations. (2024), 2024.
- [9] In Gim, Zhiyao Ma, Seung seob Lee, and Lin Zhong. Pie: A programmable serving system for emerging llm applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP ’25)*, 2025. URL <https://arxiv.org/abs/2510.24051>.
- [10] Ping Guo, Chenyu Zhu, Siyuan Chen, Fei Liu, Xi Lin, Zhichao Lu, and Qingfu Zhang. Evo-engineer: Mastering automated cuda kernel code evolution with large language models. *arXiv preprint arXiv:2510.03760*, 2025. URL <https://arxiv.org/abs/2510.03760>.
- [11] Charles Hong, Sahil Bhatia, Alvin Cheung, and Yakun Sophia Shao. Autocomp: Llm-driven code optimization for tensor accelerators, 2025. URL <https://arxiv.org/abs/2505.18574>.
- [12] Jaber Jaber and Osama Jaber. AutoKernel: Autonomous gpu kernel optimization via iterative agent-driven search, 2026. URL <https://arxiv.org/abs/2603.21331>.

- [13] Shashank Kirtania, Naman Gupta, Priyanshu Gupta, Sumit Gulwani, Arun Iyer, Suresh Parthasarathy Iyengar, Arjun Radhakrishna, Sriram K. Rajamani, and Gustavo Soares. STACKFEED: Structured textual actor-critic knowledge base editing with feedback. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 2588–2606, Suzhou, China, November 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-industry.176. URL <https://aclanthology.org/2025.emnlp-industry.176/>.
- [14] Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. The AI CUDA engineer: Agentic CUDA kernel discovery, optimization and composition. <https://sakana.ai/ai-cuda-engineer>, 2025. Sakana AI Blog Post. Accessed: May 19, 2025.
- [15] Gang Liao, Hongsen Qin, Ying Wang, Alicia Golden, Michael Kuchnik, Yavuz Yetim, Jia Jiunn Ang, Chunli Fu, Yihan He, Samuel Hsia, Zewei Jiang, Dianshi Li, Uladzimir Pashkevich, Varna Puvvada, Feng Shi, Matt Steiner, Ruichao Xiao, Nathan Yan, Xiayu Yu, Zhou Yu, Wenyuan Chi, Barney Huang, Sean Zhang, Noah Weller, Zach Marine, Wyatt Cook, Carole-Jean Wu, and Gaoxiang Liu. Kernelevolve: Scaling agentic kernel coding for heterogeneous ai accelerators at meta. *arXiv preprint arXiv:2512.23236*, 2025. URL <https://arxiv.org/abs/2512.23236>.
- [16] Edward Lin, Sahil Modi, Siva Kumar Sastry Hari, Qijing Huang, Zhifan Ye, Nestor Qin, Fengzhe Zhou, Yuan Zhang, Jingquan Wang, Sana Damani, et al. SOL-ExecBench: Speed-of-light benchmarking for real-world GPU kernels against hardware limits, 2026. URL <https://arxiv.org/abs/2603.19173>.
- [17] Hannah Lin, Martin Maas, Maximilian Roquemore, Arman Hasanzadeh, Fred Lewis, Yusuf Simonson, Tzu-Wei Yang, Amir Yazdanbakhsh, Deniz Altinbükten, Florin Papa, Maggie Nolan Edmonds, Aditya Patil, Don Schwarz, Satish Chandra, Chris Kennelly, Milad Hashemi, and Parthasarathy Ranganathan. Eco: An llm-driven efficient code optimizer for warehouse scale computers. *arXiv preprint arXiv:2503.15669*, 2025. URL <https://arxiv.org/abs/2503.15669>.
- [18] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [19] Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. *arXiv preprint arXiv:2406.11695*, 2024. URL <https://arxiv.org/abs/2406.11695>. Submitted June 17, 2024; last revised October 6, 2024.
- [20] Anne Ouyang, Simon Guo, and Azalia Mirhoseini. Kernelbench: Can llms write gpu kernels?, 2024. URL <https://scalingintelligence.stanford.edu/blogs/kernelbench/>.
- [21] Filip Petrovič, David Štrélák, Jana Hozzová, Jaroslav Olha, Szymon Bak, Vladimír Goliáš, and Gianluca Palermo. A benchmark set of highly-efficient CUDA and OpenCL kernels and its dynamic autotuning with kernel tuning toolkit. *Future Generation Computer Systems*, 108: 161–177, 2020. doi: 10.1016/j.future.2020.02.069. URL <https://doi.org/10.1016/j.future.2020.02.069>.
- [22] Dezhi Ran, Shuxiao Xie, Mingfang Ji, Ziyue Hua, Mengzhou Wu, Yuan Cao, Yuzhe Guo, Yu Hao, Linyi Li, Yitao Hu, and Tao Xie. KernelBand: Boosting llm-based kernel optimization with a hierarchical and hardware-aware multi-armed bandit, 2025. URL <https://arxiv.org/abs/2511.18868>.
- [23] Jaime Sevilla, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos. Compute trends across three eras of machine learning. In *2022 international joint conference on neural networks (IJCNN)*, pages 1–8. IEEE, 2022.
- [24] Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent, 2025. URL <https://github.com/algorithmicsuperintelligence/openevolve>.

- 389 [25] Jubi Taneja, Avery Laird, Cong Yan, Madan Musuvathi, and Shuvendu K Lahiri. Llm-vectorizer:
390 Llm-based verified loop vectorizer. In *Proceedings of the 23rd ACM/IEEE International*
391 *Symposium on Code Generation and Optimization*, pages 137–149, 2025.
- 392 [26] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler
393 for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International*
394 *Workshop on Machine Learning and Programming Languages*, pages 10–19. ACM, 2019. doi:
395 10.1145/3315508.3329973.
- 396 [27] Laura Wang et al. Kernelfalcon: Deep agent architecture for autonomous gpu kernel generation.
397 PyTorch Blog, November 2025. Blog post.
- 398 [28] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and
399 James Zou. Textgrad: Automatic" differentiation" via text. *arXiv preprint arXiv:2406.07496*,
400 2024.
- 401 [29] Kaixuan Zhang, Chutong Ding, Shiyu Qian, Luping Wang, Jian Cao, Guangtao Xue, Cheng
402 Huang, Guodong Yang, and Liping Zhang. WaveTune: Wave-aware bilinear modeling for
403 efficient gpu kernel auto-tuning, 2026. URL <https://arxiv.org/abs/2604.10187>.
- 404 [30] Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. Cudaforge:
405 An agent framework with hardware feedback for cuda kernel optimization. *arXiv preprint*
406 *arXiv:2511.01884*, 2025.
- 407 [31] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida
408 Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. An-
409 sor: Generating high-performance tensor programs for deep learning. In *Proceedings of the*
410 *14th USENIX Symposium on Operating Systems Design and Implementation*, pages 863–879.
411 USENIX Association, 2020.

412 Appendix

413 A Additional Ablation Studies

414 A.1 Impact of Learning on Performance

415 To understand the impact of KERNELBLASTER’s *Knowledge Base* on code optimization performance,
 416 we conduct a study by limiting the ability of KERNELBLASTER’s MAIC-RL flow to learn from
 417 experience effectively. As described in Section 4.5, it is critical to develop and explore a wide range
 418 of optimization strategies when optimizing a diverse codebase. However, this diversity relies on
 419 KERNELBLASTER’s ability to mutate its *Knowledge Base* by modifying and expanding entries.

420 We provide an example of states within the *Knowledge Base* in Figure 7a. First, we study the impact
 421 of developing a *Knowledge Base* from scratch, compared to iterating upon a partially-trained data
 422 structure. The comparison between Figure 6a and Figure 6b shows the rate at which optimizations
 423 are applied while optimizing the KernelBench Level 1 dataset. As seen in the left plot, although the
 424 first round to construct a *Knowledge Base* is expensive, future optimization passes can benefit from
 425 much faster coverage of optimizations. Furthermore, these generated databases can be reused across
 426 scenarios; The right plot demonstrates how a *Knowledge Base* trained on an A6000 GPU can be used
 427 for optimization runs across different GPU platforms.

428 To understand the impact of the long-term memory (*Knowledge Base*), we isolate the contribution
 429 of profiling versus memory reuse by comparing against a `no_mem_agent` that has access to full
 430 Nsight Compute profiling but operates with an empty knowledge base and no state-conditioned
 431 reuse. `no_mem_agent` underperforms our full system, achieving 1.67x slower results. Thus, profiling
 432 information alone provides meaningful but limited gains, whereas persistent knowledge reuse is
 433 necessary to transfer previously successful optimization strategies across kernels. Ablations show that
 434 profiling feedback alone is necessary but not sufficient: the strongest gains arise from the interaction
 435 between structured profiling signals and a persistent, state-aware knowledge base.

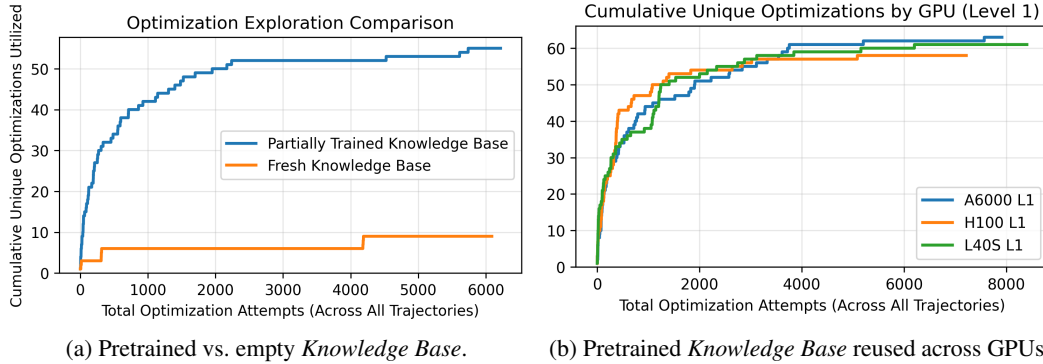
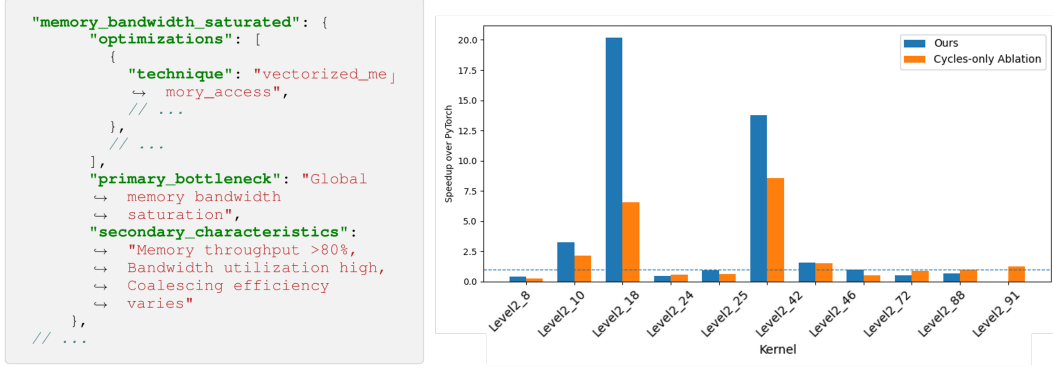


Figure 6: Reusable persistent memory: (a) discovery and application of new optimizations with a pretrained vs. empty *Knowledge Base*; (b) the same *Knowledge Base* trained on an A6000 transfers across other NVIDIA GPU platforms.

436 A.2 Impact of Semantic Profiling Data on Agent Performance

437 To understand the impact of profiling fidelity, we conducted ablation studies against an Actor-Tactician
 438 agent that only receives latency rewards instead of detailed NCU profiles. The cycle-only agent
 439 underperforms approaches that use NCU (1.22x speedup over PyTorch on Level-2 compared to 1.57x
 440 with NCU data, shown in Figure 7b), indicating that scalar latency alone is insufficient to infer why
 441 a kernel is slow or which optimization direction to pursue. Furthermore, in addition to limiting
 442 performance in the inner Actor-Tactician loop, stripping semantic data prevents the Critic-Strategist
 443 from effectively learning from past experience.



(a) A sample of the *Knowledge Base* data (b) Speedup over PyTorch: full text-based reward vs. a cycles-only structure, specifying discovered states.

Figure 7: Memory contents and text-based reward ablation. (a) The learned memory does not collapse to a single transformation family: no state exceeds 20% of applications, justifying state-conditioned retrieval. (b) Replacing text-based critiques with cycle-only scalar feedback degrades optimization across most kernels ($1.22\times$ vs. $1.57\times$ on Level 2), confirming that language carries information scalar timing cannot.

A.3 Comparison Against Minimal Agent

Finally, we compare against a minimal agent. At each iteration, this agent directly takes in CUDA code and NCU profiling data and outputs optimized code. When running 10 trajectories of length 10, this agent requires 2.4x tokens compared to KERNELBLASTER. We observe this is due to two primary causes:

1. Due to not having access to a *Knowledge Base* and a guided reasoning process, this agent must devote more tokens up-front for reasoning about the optimizations
2. The minimal agent requires more retrievals for correctness compared to KERNELBLASTER.

Compared to KERNELBLASTER, this minimal agent has 0.379x performance improvement per token. Furthermore, KERNELBLASTER achieves better performance in 71% of cases compared to the simplified agent.

To understand the impact of the long-term memory (*Knowledge Base*), we isolate the contribution of profiling versus memory reuse by comparing against a `no_mem_agent` that has access to full Nsight Compute profiling but operates with an empty knowledge base and no state-conditioned reuse. `no_mem_agent` underperforms our full system, achieving 1.67x slower results. Thus, profiling information alone provides meaningful but limited gains, whereas persistent knowledge reuse is necessary to transfer previously successful optimization strategies across kernels. Ablations show that profiling feedback alone is necessary but not sufficient: the strongest gains arise from the interaction between structured profiling signals and a persistent, state-aware knowledge base.

B Additional Analysis

B.1 Search Breadth and Search Depth

The key search hyperparameters are trajectory count and trajectory length. Increasing trajectory count improves exploration, but we observe diminishing returns beyond eight trajectories for median and upper-quartile kernels. Lower-quartile kernels continue to benefit from additional breadth, suggesting that difficult kernels require broader exploration. Increasing trajectory length shows similar diminishing returns beyond four transformations for most kernels, although high-upside kernels can still benefit from longer sequences.

These results support the actor-critic structure. The actor needs enough breadth to explore transformations, while the critic should decide when additional rollouts are unlikely to pay off. This prevents the method from becoming an unbounded

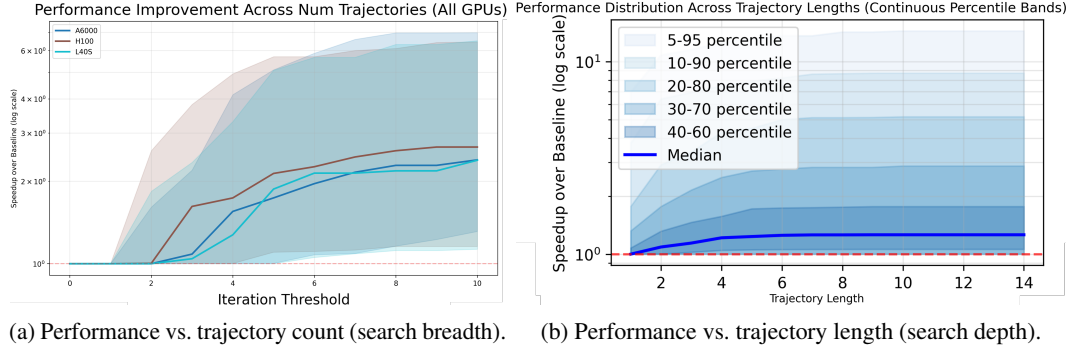


Figure 8: The impact of hyperparameter tuning on search dynamics: shaded regions show the inter-quartile range of optimized kernels. Breadth and depth both exhibit diminishing returns.

B.2 Correctness, Verification, and Cheating Prevention

To prevent shortcut behaviors such as calling PyTorch or emitting non-vanilla-CUDA code, we employ a soft-verification agent that compares the optimized kernel against the original KernelBench baseline. In addition to our test harness checking numerical correctness, we use an LLM agent that detects reward hacking (such as eliminating or moving operations out of profiled regions) and calling external optimized libraries, directly addressing concerns about cheating in agent-generated kernels. Finally, we verify verification agent’s robustness by measuring its agreement with expert human reviewers.

However, the protocol does not constitute a proof of behavioral equivalence and has several limitations that should be acknowledged. First, the verification agent is itself an LLM and inherits the underlying model’s reliability characteristics: it can produce both false positives (rejecting legitimate optimizations whose surface form resembles a known cheating pattern, such as algebraic simplifications that genuinely eliminate redundant operations) and false negatives (missing novel evasion strategies that do not match the prompt’s enumerated cheat templates). Second, the cheat-pattern catalog is anchored to the KernelBench/CUDA setting (e.g., dispatching to PyTorch, calling cuDNN/cuBLAS, hoisting operations out of the timed region); the OpenCL/Adreno and RVV/RISC-V backends use the same conceptual checks, but their cheat patterns have not been independently audited and may differ. Fourth, compiler-level shortcuts are outside the agent’s view: if the emitted CUDA is structured such that NVCC, the driver, or a vendor library substitutes a fast-path implementation internally, the verification agent does not observe that substitution.

Finally, a stronger protocol would combine LLM-level pattern detection with deterministic static analysis (e.g., checking the call graph for forbidden symbols) and randomized differential testing across broader input distributions. We document these limitations explicitly so that the verification step is auditable rather than represented as eliminating reward hacking entirely.

B.3 Optimization Distribution on Secondary HW Targets

In addition to characterization the optimization distribution on the primary NVIDIA GPU targets, we also provide a summary of the optimization distribution on Qualcomm targets in Figure 9. Compared to NVIDIA GPUs, we observe a smaller set of optimizations dominating total optimization usage. This is due to the fact that the OpenCL compilation process requires greater hand tuning, whereas certain optimizations can be implemented directly by the CUDA compilation process. Additionally, we observe certain CUDA specific optimizations selected in the Qualcomm flow (such as tensor core utilization). While these are initially discovered and attempted, they do not result in code with improved performance, and no longer dominate optimization attempts.

B.4 Optimization/State Clustering Analysis

We use the trajectories stored in KERNELBLASTER’s *Knowledge Base* to validate the state abstraction used by MAIC-RL. Each successful CUDA action is represented by its pre-application profiler state, a 15-dimensional Nsight Compute bottleneck signature covering memory and compute throughput,

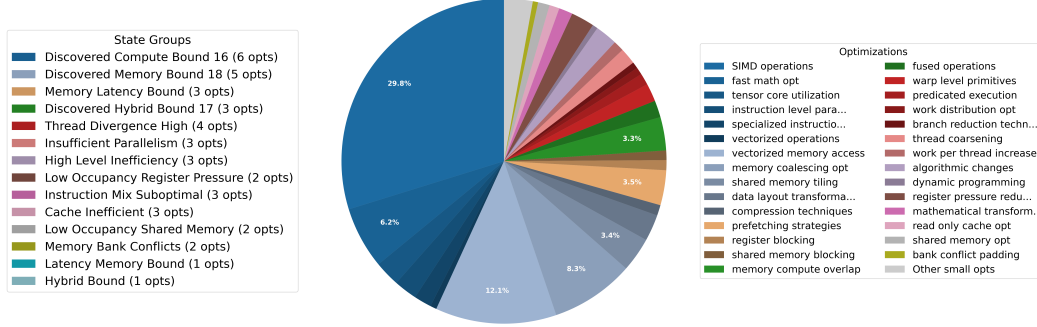


Figure 9: Distribution of Qualcomm optimization applications across performance states (Kernel-Bench L1, Adreno 650 GPU).

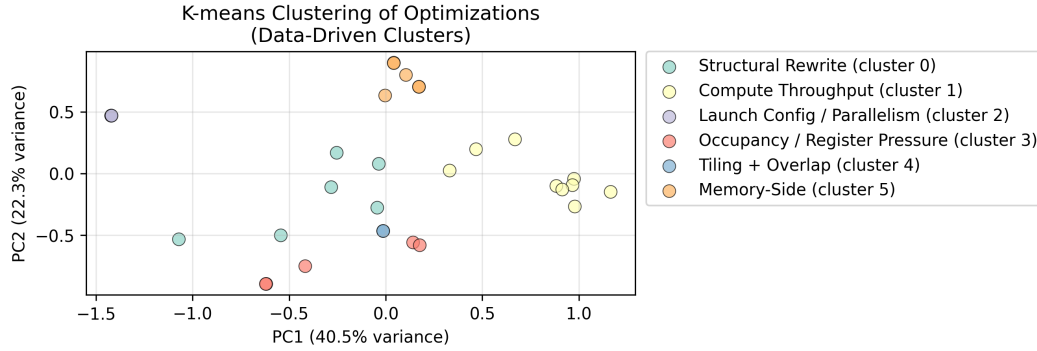


Figure 10: Synthetic Clustering of Top Optimizations

occupancy, cache behavior, pipeline utilization, and warp scheduling. K-means clustering over these signatures recovers six interpretable action regimes: memory-side fixes, launch/parallelism fixes, occupancy and register-pressure interventions, tiling with compute-memory overlap, instruction-level compute tuning, and structural rewrites. This state-conditioned view is substantially more informative than clustering actions by usage frequency or binary success vectors, which mostly separate outliers from the rest. The clusters also align with the *Knowledge Base*'s manual state taxonomy: memory-bandwidth states select memory-side actions, compute-throughput states select instruction-level tuning, balanced memory-compute states select tiling/overlap, and insufficient-parallelism states select launch-configuration changes. The only heterogeneous regime is structural rewrite, whose pre-action signatures are saturated across many metrics, consistent with its role as a last-resort intervention. This analysis suggests that KERNELBLASTER should retrieve and update actions through profiler-derived macro-states, while allowing the critic to refine or merge finer manual states when trajectory evidence does not support them.

B.5 Cost and Efficiency

Because token prices vary across model, date, and currency, we report token usage directly. On KernelBench Level 1 and Level 2 using gpt-5.1, KERNELBLASTER consumes geometric means of 2,297,652 and 1,600,997 tokens per problem, respectively. Token count varies with code size, number of profiled kernels, and optimization complexity. The more meaningful metric for this paper is not raw speedup alone, but speedup per persistent learning signal: each rollout contributes scalar outcomes and textual critiques that can be reused by later tasks.

B.6 Limitations

KERNELBLASTER's system also exposes limitations for future work. Full-model optimization requires hierarchical decomposition because whole-network code generation introduces long boilerplate and heterogeneous performance states. Cross-target optimization requires stronger profiling